

EM Services — Admin Section Feature Overview

UC-011 Cost Analytics Dashboard & UC-012 Vendor Account Lifecycle · Lift Inspection & Estate Defect Management System · July 2026

1 · Admin shell & access control

Component	What it does
AdminLayout.jsx	Dedicated admin chrome: EM Services brand header, nav tabs (Cost Analytics → /admin/costs, Vendors → /admin/vendors), notification bell placeholder, avatar menu (profile / logout). Nav collapses into a hamburger drawer on mobile.
Role-based routing	RoleLayout reads the signed-in profile and renders the admin chrome only for role = admin. Both pages sit behind ProtectedRoute (Supabase auth token required).
Triple role gating	1) Layout only rendered for admins · 2) each page has its own role check that shows an error for non-admins · 3) every backend route enforces requireRole('admin').

2 · UC-012 Vendors page — user-facing features

Feature	Details	Why it matters
Accountable onboarding	Dialog capturing company details, brands serviced, contract start/end, contract document upload (PDF/DOCX → Cloudinary), plus accountability fields: account holder name, job title, access reason. Creates the vendor's login (Supabase auth) with an admin-set password.	Every external account is traceable to a named person with a stated reason — an audit/governance requirement, not just a form.
Credential helpers	One-click strong password from the browser's cryptographic RNG (16 chars); login email auto-suggested from holder name + company domain (e.g. sing.en@dymatics.com).	Removes weak-password risk and typo-prone manual entry.
Expiry-sorted vendor list	Vendors listed soonest-expiring first with countdown chips: green (healthy), amber (≤ 30 days / today), red (expired). Search box and CSV export included.	The next contract that needs attention is always at the top.
Renew	Confirmation dialog; new contract end defaults to +1 year; optional replacement contract document. Renewing a suspended vendor reactivates its login.	Matches the real annual servicing cycle; one action fixes both contract and access.
Suspend	Early termination with confirmation; immediately blocks the vendor's login.	Contract-bounded access can be revoked on the spot.
Edit details / audit history	Non-contract details editable in place; per-vendor history dialog shows the full audit trail of lifecycle actions.	Every change is recorded and reviewable.
Run expiry check now	Button that triggers the same suspension logic as the nightly job, with live Socket.IO updates to the page.	Demo-friendly and useful when an admin can't wait for 1 AM.

3 · UC-012 backend — API endpoints (/api/admin/vendors)

Endpoint	Auth	Purpose
POST /	admin	Onboard vendor (multipart, optional contract_doc); creates auth account.
GET /	admin	Vendor list sorted by contract_end ascending.
GET /expiry-check	CRON_SECRET	Nightly job: suspends vendors whose contract_end passed but whose account is still active; notifies admins via Socket.IO.
POST /run-expiry-check	admin	Same logic, triggered on demand from the vendor page.
POST /:id/renew	admin	New contract_end (+ optional new document); reactivates a suspended account.

Endpoint	Auth	Purpose
POST <code>/:id/suspend</code>	admin	Early termination; blocks login.
PATCH <code>/:id</code>	admin	Edit non-contract details (holder, contact...).
GET <code>/:id/history</code>	admin	Vendor audit trail.

4 · UC-012 automation — scheduled enforcement

Item	Details
GitHub Actions workflow	<code>.github/workflows/contract-expiry-check.yml</code> — daily cron at 01:00 SGT calling <code>GET /api/admin/vendors/expiry-check</code> with the <code>CRON_SECRET</code> bearer token. Named explicitly in the High-Level Design (§ scheduled layer).
Why it exists	<code>contract_end</code> is just a date column; this job is what actually enforces it — flipping the linked user account to suspended the night the contract lapses.
Current status	Failing (expected): the backend isn't deployed to Render yet, so the <code>RENDER_BACKEND_URL</code> and <code>CRON_SECRET</code> repository secrets aren't set. Fix at deploy time: add both secrets in GitHub → Settings → Secrets → Actions. Until then the workflow can be disabled from the Actions tab to stop failure emails (no code change needed).

5 · UC-011 Cost Analytics page — what's on it (top to bottom)

Section	What the admin sees	How it's computed
KPI tiles (×4)	Total maintenance spend (+ job count) · Spend movement vs prior period with red/green arrow · Projected exposure from active AI risk alerts · Average cost per job.	Live aggregation of the filtered job rows. Movement compares the window against the equally long window immediately before it (boundary-safe — no double counting).
Filter bar	Quick-range chips (Last 30 / 90 days, This year, All time), From/To dates with validation, Block / Category / Contractor dropdowns, per-filter clear and Clear-all chip.	Filters persist in the URL, so any filtered view is bookmarkable and shareable. Every panel recomputes on change.
Insights panel	3–5 auto-written plain-English sentences summarising the current view: headline spend + movement, contractor concentration, costliest category + biggest riser, most urgent lift, next-month projection. Rewrites live as filters change.	Deterministic rules engine — each sentence is a pure function with a 'worth saying' threshold (e.g. concentration only speaks at $\geq 40\%$ share). No LLM: no API key, no latency, no hallucination, fully unit-tested.
Spend by category (bar)	Cost per category, cost-heaviest first; tooltip shows spend and job count. Clicking a bar drills down (sets that category as the active filter).	Chart.js bar over grouped filtered rows; same drill-down pattern as the manager heatmap.
Monthly spend trend (line)	Solid line of monthly actuals; dashed amber 3-month projection with a shaded likely range; a Backtested accuracy badge (hover for the month-by-month audit).	See § 6 — the predictive engine.
Lift watchlist — repair vs replace	Lifetime spend per lift vs the \$8,000 replacement-review threshold: progress bar, % of threshold, estimated time to review ("~4 mo" / "now"), status chip (Healthy / Approaching review / Review replacement).	Lifetime totals ignore date filters by design (the whole life counts). Time-to-review projects each lift's average spend rate over its last 6 complete months.
Cost per contractor	Sortable table: jobs, actual cost, avg cost/job, share of spend, and a 'Vs peers' price benchmark chip (e.g. +28% on Doors) with tooltip.	Benchmark compares avg cost/job within the same category only, so job mix can't confound it; flags need ≥ 2 own jobs, ≥ 2 peer jobs and $\geq 15\%$ deviation.
Recent costed jobs	Latest close-outs (showing 10 of N) with date, block, category, contractor, cost — and an Outlier chip on any job over 2× its category average.	The drill-down behind every aggregate; Export CSV downloads the full filtered job list.

6 · The predictive & analytical engine (inside costService.js)

Capability	Method	Honesty guards
Next-month spend projection	Damped-trend exponential smoothing (Holt's method, $\phi = 0.85$) — the textbook short-horizon forecasting technique. Projects 3 months; damping makes the curve flatten instead of extrapolating in a straight line forever.	Fitted on COMPLETE months only (a half-finished month would drag the trend down). Refuses to project with under 3 complete months. Clamped at \$0.
Uncertainty band	$\pm 1.28\sigma$ ($\approx 80\%$) band from the model's own one-step-ahead errors, widening with the square root of the horizon.	The band is derived from how wrong the model has actually been — not decoration.
Backtested accuracy badge	Walk-forward validation: for each historical month, the model is re-fitted on data strictly before that month and graded against the real total. Badge shows mean absolute % error; tooltip lists every prediction vs actual.	No peeking (proven by test: a June spike cannot influence June's own prediction). Skips zero months; badge turns amber/red if the model is mediocre; disappears rather than fake a score when history is too thin.

Capability	Method	Honesty guards
Spend movement	Current filtered window vs the equally long window immediately before it.	Prior window ends the day before the current one starts — a job on the boundary date is never counted twice (regression-tested).
Top mover	Largest month-on-month category increase, quoted inside the Insights panel.	Compares the last two COMPLETE months — a partial month can't fake a movement.
Lift time-to-review	$(\text{Threshold} - \text{lifetime spend}) \div \text{average monthly spend}$ over the last 6 complete months.	Shows '—' when there's no recent spend to project from; 'now' when already past.
Price benchmarking	Per-contractor avg cost/job vs the job-weighted peer average within each category; the largest significant deviation is flagged.	Within-category only (controls for job-mix confounding); minimum-evidence rules (≥ 2 own jobs, ≥ 2 peer jobs, $\geq 15\%$) before anything is claimed.

7 · Data layer, testing & current status

Item	Details
Architecture	All analytics live as pure, exported functions in costService.js operating on raw job-level rows shaped exactly like the backend's inspections + ai_predictions records (HLD § 6.10 response shapes). The page only calls thin async wrappers.
Data source (current)	Mock dataset (costMocks.js): 36 closed costed jobs across 7 months, 4 blocks, 6 categories, 4 contractors, with lift codes, plus 4 active AI cost predictions. Every number on the page is genuinely computed from these rows at runtime — nothing is canned.
Swap to live data	When the UC-011 backend endpoints (/api/admin/costs/*) are built, only the data source inside each wrapper changes; shapes, page and tests stay identical.
Test suite	45 passing Vitest tests with hand-checkable fixtures covering: filter inclusivity, variance windows (incl. the boundary case), grouping, trend gap-fill, forecast properties (flat → flat, rising → damped rise, partial-month exclusion, zero clamp, band widening), backtest no-peeking, top-mover rules, benchmark guards, insight sentence thresholds, watchlist maths. The audit also caught and fixed two real bugs: a window-boundary double count and timezone-fragile month arithmetic.
Deployment dependencies	1) UC-011 backend endpoints (assigned to Davian) → flip the service to live data. 2) Render deployment + GitHub secrets → contract-expiry workflow goes green. 3) Visual once-over with a real admin login (blocked locally: no valid admin account in the connected Supabase project).

Honest demo framing: every calculation, filter, projection and test is real and runs live; the input data is a demo dataset until the cost API is deployed. The forecasting is statistics (damped exponential smoothing), not machine learning — and the dashboard displays its own backtested error rate so nobody has to take its word for it.